

SCHEMA DRIVEN FRONTEND FORM GENERATOR WITH ACCESSIBILITY COMPLIANCE

25-26J-436

Project Proposal Report
Thihansi Sandanaki Gunawardena

B.Sc. (Hons) in Information Technology, Specializing in
Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

SCHEMA DRIVEN FRONTEND FORM GENERATOR WITH ACCESSIBILITY COMPLIANCE

25-26J-436

Project Proposal Report
Thihansi Sandanaki Gunawardena

B.Sc. (Hons) in Information Technology, Specializing in
Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

Declaration page of the candidates & supervisor

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Gunawardena T S	IT22350114	

The above candidate is carrying out research for the undergraduate dissertation under my supervision.



Signature of the Supervisor

27/08/2025

Date



Signature of the Co-Supervisor:

29/08/2025

Date

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Mr. Jeewaka Perera, and co-supervisor, Ms. Thilini Jayalath, for their continuous guidance, encouragement, and invaluable support throughout the progress of this research. Their expertise, constructive feedback, and constant motivation have been instrumental in shaping my understanding and in helping me refine my work to meet higher academic standards. I am deeply appreciative of the time and commitment they have dedicated to mentoring me during this project.

I am also thankful for the thoughtful suggestions and feedback that have not only improved the technical aspects of my work but also provided me with the confidence to approach challenges with a broader perspective. The encouragement I received from both supervisors has been a source of inspiration, and I am truly grateful for their patience and support.

With their mentorship, I am motivated to continue this project with dedication and commitment, and I look forward to producing work of quality and significance that reflects the trust and guidance they have placed in me.

Abstract

Modern web applications rely heavily on form-based interactions for collecting and processing data. Ensuring that these forms are generated consistently, validated against backend rules, and accessible to all users is essential for usability, accuracy, and compliance. Existing tools such as Formik and React Hook Form [1], [2] and schema-driven libraries such as JSONForms [3] provide partial solutions, but they often require manual configuration, duplicate validation logic, and lack built-in accessibility assurance. Research efforts including pix2code [4], Prototype2Code [5], and Diffusion UI [6] have demonstrated automated code generation and AI-driven interface design, while accessibility-focused approaches such as AltIcon [7] provide alt-text support for icons. However, these contributions remain isolated and do not offer an integrated solution.

This project proposes a schema-driven frontend form generator with a visual builder that unifies code generation, validation, and accessibility in a single workflow. The system generates React form code directly from JSON Schema while synchronising validation rules with backend data transfer objects from Spring Boot. A drag-and-drop visual interface enables non-technical users to design forms, and accessibility is automated through embedded WCAG 2.1 compliance checks, ensuring that generated forms include semantic labels, ARIA attributes, and error identification by default.

The proposed system aims to reduce duplication of validation logic, improve development efficiency, and enhance inclusivity in form-driven applications. Evaluation will consider functional accuracy, accessibility compliance, usability of the visual builder, and performance in generating forms of varying complexity. The outcomes are expected to contribute towards a more reliable, user-friendly, and accessible approach to frontend form development.

Keywords: Form generation, JSON Schema, React, Backend validation, Visual builder, WCAG 2.1, Accessibility automation.

Table of Contents

Declaration page of the candidates & supervisor.....	i
Acknowledgement	ii
Abstract.....	iii
Table of Contents	iv
List of Tables	vi
List of Figures.....	vi
1. INTRODUCTION.....	1
1.1. Background.....	1
1.2. Literature Survey	3
1.3. Research Gap.....	5
1.4. Research Problem	8
2. OBJECTIVES	9
2.1. Main Objective	9
2.2. Specific Objectives.....	9
3. METHODOLOGY.....	10
3.1. System Overview	10
3.2. Core Processes and System Inputs	11
3.3. Tools, Frameworks, and Technologies	12
3.4. Evaluation and Testing Approach.....	13
3.5. Expected Outcomes	13
4. Functional and Non-Functional Requirements	14
4.1. Functional Requirements	14
4.2. Non-Functional Requirements	15
5. GANTT CHART	16

6.	WORK BREAKDOWN STRUCTURE	17
7.	COMMERCIALISATION AND ENTREPRENEURSHIP POTENTIAL	18
7.1.	Market Review and Target Users	18
7.2.	Value Proposition	18
7.3.	Business Model Approach	18
7.4.	Cost and Resource Structure	19
7.5.	Commercial Viability Factors	19
7.6.	Marketing and Adoption Strategy	19
8.	BUDGET AND BUDGET JUSTIFICATION	20
9.	REFERENCES	21

List of Tables

Table 1.1 - Comparison of existing research and tools against the proposed system..	7
Table 8.1 - Budget Justification	20

List of Figures

Figure 3.1 - System Overview Diagram	10
Figure 3.2 - Component System Overview Diagram.....	11
Figure 4.1 - Gantt Chart for Schema-Based Frontend Builder	16
Figure 5.1 - Work Breakdown Structure for Schema Based Frontend Builder	17

1. INTRODUCTION

1.1. Background

Modern web applications need user interfaces that are not only efficient but also secure and accessible. Forms are one of the main ways these applications collect and process information from users. As a result of it the way forms are generated and validated is very important for accuracy, usability, and compliance. The proposed research system aims to provide a single platform where every part of the process happens together. From schema definition to backend validation, and from frontend code generation to accessibility checks, the entire cycle is automated and managed in one place. This integration is what makes the system different from existing tools, which usually handle only one part of the process in isolation. Within this platform, the Frontend Generator and Visual Builder module focuses on generating validated React code, enabling visual drag-and-drop form creation, integrating AI-driven optimizations, and embedding WCAG 2.1 compliance automation.

Nowadays, developers often rely on third-party libraries to implement forms in modern JavaScript frameworks. Popular tools such as Formik [1] and React Hook Form [2] provide declarative methods to manage form state and validation in React. Schema-driven libraries like JSONForms [3] and Uniforms [8] allow form structures to be defined using JSON Schema, which improves consistency and reusability. Even though these approaches are effective, they have persistent challenges. Typically they need a considerable amount of manual configuration, often leading to duplication of validation logic between frontend and backend services. Furthermore, they don't have any built-in accessibility assurance mechanisms, so developers are left to decide whether to comply with WCAG 2.1 [9].

At the backend level, tools such as the OpenAPI Generator [10] demonstrate how specifications can be transformed into code artifacts automatically, reducing repetitive tasks. However, frontend equivalents of such code generators remain limited in scope and functionality. They generally fail to synchronize validation rules with backend DTOs, creating potential mismatches that can compromise data integrity.

Accessibility is another area where existing solutions are lacking. Even though the Web Content Accessibility Guidelines (WCAG) 2.1 [9] and its companion documentation [11] provide detailed requirements for accessible forms and interfaces, manual implementation of these guidelines remains a challenge for development teams. Automated support for accessibility metadata, alternative text, semantic roles, and error identification is minimal in current tools, which limits inclusivity and increases the risk of non-compliance.

Development of a frontend generator that integrates schema-driven code generation, validation synchronization with backend services, non-technical drag-and-drop visual building, AI-assisted optimization, and accessibility compliance automation is highly motivated in light of these drawbacks. This combination offers the groundwork for better user experience, wider compliance with international accessibility standards, and effective form development.

1.2. Literature Survey

Research on automated form and interface generation has developed along several directions, including schema-driven approaches, deep learning for layout recognition, accessibility support, recommendation systems, and secure validation. While these studies demonstrate important advances, most of them focus on a single dimension of the problem. There is limited work that integrates frontend generation, backend validation, accessibility compliance, and intelligent optimization within one platform.

Visual Builders and Schema-Driven Code Generation

Early research introduced the idea of end-to-end GUI-to-code translation from visual designs. Systems such as pix2code [4] and neural machine translation models for GUI skeleton generation [12] showed that screenshots or design images could be automatically converted into code. However, these methods were constrained by accuracy issues, limited datasets, and outputs that lacked responsiveness or accessibility features. Later works improved code fidelity by applying convolutional networks to classify components [13] or mapping elements across platforms [14], but they remained tied to Android, iOS, or static HTML. Model-driven approaches extended this line of research by introducing metamodel transformations from UI mockups into Angular or React components [15] and schema-driven dynamic form engines [16]. These contributions demonstrate that scaffolding from structured models can reduce developer effort, yet they continue to require manual handling of validation and provide minimal support for compliance with accessibility standards.

Accessibility and Multi-Modal Interfaces

Accessibility has been addressed through targeted systems designed to aid specific user groups. Voice-based form filling tools have been created to assist visually challenged users [18], and conversational agents have been proposed to support aged and monolingual citizens in completing government forms [19]. More recently, multimodal models have been applied to generate alt-text for UI icons during development, achieving high accuracy and reducing accessibility debt [7]. While these studies confirm the importance of inclusivity, they remain isolated prototypes. None

of them integrate accessibility into schema-to-code pipelines or extend compliance enforcement across entire frontend generation systems.

Form Recommendations and Intelligent Suggestions

Research has also examined how semantic and structural modeling can assist developers during form creation. Transformer-based models have been trained on large corpora of online forms to recommend questions, block types, and candidate options [20]. Similarly, form modeling approaches that use visibility matrices have been proposed to reduce redundancy by linking shared variables across tasks [21]. These methods highlight how recommendation systems can reduce manual work and improve efficiency. However, they operate independently of code generation workflows and do not address validation synchronization or accessibility automation.

Security and Compliance

Input validation and vulnerability mitigation remain central to secure form systems. Pattern-based approaches have been advocated to replace ad hoc fixes, particularly against common injection attacks [22]. More recently, security frameworks for AI-generated code have integrated prompt libraries, knowledge bases, and automated static analysis tools to systematically reduce vulnerabilities in generated applications [23]. These contributions reinforce the importance of embedding security and validation into generation pipelines. Yet they primarily focus on backend or general web applications, with little emphasis on frontend code generation or compliance with accessibility and privacy standards.

When looking across these areas of research, some common limitations become clear. Most existing approaches focus on only one part of the problem, such as code generation, layout improvement, accessibility, recommendations, or security, without bringing these strands together. Consistency between frontend generation and backend validation is rarely addressed, and accessibility standards like WCAG 2.1 are often overlooked in automated solutions. In addition, while artificial intelligence has been used to improve layouts or responsiveness, it has not yet been combined with schema-driven generation in a way that supports both usability and compliance. These gaps

highlight the need for a system that brings everything together, offering schema-to-code generation, backend synchronization, a drag-and-drop form builder, and automated accessibility support within one platform.

1.3. Research Gap

Gap 1 - Fragmentation of Existing Contributions: The review of existing studies shows clear progress in areas such as schema-driven code generation, artificial intelligence for layout optimization, accessibility support, recommendation systems, and secure validation. These works have demonstrated that it is possible to generate code from design models [16], [15] [4], [12] optimize layouts using diffusion or neural models [5], [6] , [17], support specific user groups through assistive technologies [18], [19], [7], and reduce vulnerabilities using validation frameworks [22], [23]. However, they also reveal that these contributions remain fragmented, each addressing only a single part of the overall challenge. None of the reviewed approaches provides a unified solution where form generation, backend synchronization, accessibility, security, and optimization are combined into one workflow.

Gap 2 - Lack of Frontend Backend Validation Synchronization: A major limitation lies in the lack of integration between frontend and backend validation. While schema-driven builders can create frontend forms from JSON or mockups [16], [15], [13] they do not ensure that validation logic remains consistent with backend DTOs or service definitions. On the other hand, backend-oriented frameworks [22], [23] highlight the importance of validation and security, but they do not extend this functionality into frontend code generation. This gap leaves developers to manually align validation across layers, increasing duplication and the risk of mismatches.

Gap 3 - Lack of Accessibility Support: Accessibility is another area where gaps are evident. Some studies have designed systems to assist visually impaired or monolingual users [18], [19], and others have proposed multimodal approaches to automate alt-text generation [7]. Yet these remain isolated solutions. None of them embed WCAG 2.1 compliance checking into automated form generation pipelines,

leaving accessibility as an afterthought rather than a built-in guarantee. This absence is particularly significant given the global importance of accessibility standards for web applications.

Gap 5 - Restricted Usability of Visual Builders: In addition, visual builders that exist today are primarily targeted at developers and require technical expertise [16], [15]. They are not designed to support non-technical users through simple drag-and-drop interfaces. This limits the practicality of current approaches for broader adoption in low-code or no-code environments. A research gap remains for solutions that make form generation more accessible not only for end users but also for designers or analysts without programming knowledge.

Absence of a Comprehensive End-to-End Solution: In summary existing research has made advances in each of these areas in isolation but has not yet provided a comprehensive system that combines them. There is no current solution that offers schema-to-React code generation synchronized with backend DTO validation, supported by a non-technical drag-and-drop form builder, and strengthened by automated WCAG 2.1 accessibility and security checks. This integration forms the central gap that the proposed system aims to address.

The comparison below shows the past research and industry tools contribute important advances, but none provide a complete end-to-end solution. Systems such as pix2code [4] and Prototype2Code [5] demonstrate automated code generation with AI support, while Diffusion UI [6] applies diffusion models for layout optimization. AltIcon [7] focuses on accessibility by generating alt-text for icons, and JSONForms [3] provides schema-driven generation with some visual editing support and accessible renderers. However, backend validation synchronisation is absent across all existing approaches, accessibility is limited to partial implementations, and drag-and-drop builders are either missing or developer-oriented rather than designed for non-technical users. In contrast, the proposed system integrates schema-driven React code generation, backend validation synchronisation, a non-technical visual builder, and automated WCAG 2.1 compliance in a single workflow, addressing a clear and unmet gap in the literature.

Feature	pix2code [4]	Prototype 2Code [5]	Diffusion UI [6]	AltIcon [7]	JSONForms [3]	Proposed System
Schema-driven Code Generation	✓	✓	✓	✗	✓	✓
Backend Validation Synchronisation	✗	✗	✗	✗	✗	✓
Drag-and-Drop Builder	✗	✗	✗	✗	✓	✓
Accessibility (WCAG 2.1)	✗	✗	✗	✓	✗	✓

Table 1.1 - Comparison of existing research and tools against the proposed system

1.4. Research Problem

Form generation and validation are central to the reliability and usability of modern web applications. Yet existing approaches treat different aspects of the process in isolation. Frontend code is often generated or scaffolded from schemas [16], [15], [4] but the resulting forms require developers to manually duplicate or adjust validation rules to match backend data transfer objects, increasing the likelihood of inconsistency. Accessibility has been studied through targeted solutions such as voice-based form filling [10], [11] or icon-level alt-text generation [12], but the automation of standards such as WCAG 2.1 within frontend code generation pipelines is almost entirely absent. Artificial intelligence has been applied to improve layouts and responsiveness [7]-[9], yet its integration into schema-driven generation workflows is still unexplored. Visual builders and schema-driven engines [16], [15] improve productivity but remain developer-oriented, offering limited usability for non-technical users.

These limitations give rise to several problems that guide this research. How can frontend React form generation be synchronized with backend validation schemas in a way that reduces duplication and ensures consistency? How can accessibility guidelines such as WCAG 2.1 be embedded into automated form generation rather than applied as a separate step? What role can artificial intelligence play in optimizing generated forms so that they not only follow schemas but also improve usability and compliance? Finally, how can a visual form builder be designed so that non-technical users can create and adjust forms without programming knowledge?

The research problem addressed in this proposal is therefore the absence of an integrated solution that brings together schema-to-React code generation, backend validation synchronization, accessibility compliance automation, and a user-friendly drag-and-drop builder. This problem forms the foundation for the objectives of the proposed research.

2. OBJECTIVES

2.1. Main Objective

The overall objective of the research project is to design and develop an integrated form generation platform that automates schema-to-code workflows, synchronizes frontend and backend validation, and ensures usability, security, and accessibility in modern web applications. Within this platform, the Frontend Generator and Visual Builder section specifically aims to generate validated React code, provide a user-friendly drag-and-drop interface for form creation, embed accessibility compliance checks, and explore the use of artificial intelligence for layout and usability optimization.

2.2. Specific Objectives

- i. Generate validated React form code from JSON Schema definitions
 - Ensure that generated forms follow schema constraints accurately.
 - Provide support for frontend form validation rules derived directly from backend data transfer objects.
- ii. Synchronize frontend validation with backend services
 - Maintain consistency between frontend and backend validation to reduce duplication and prevent mismatches.
 - Support interoperability with backend implementations in Spring Boot.
- iii. Develop a non-technical drag-and-drop visual form builder
 - Enable non-technical users to design and configure forms without programming knowledge.
 - Provide a low-code/no-code environment to improve accessibility for wider user groups.
- iv. Automate WCAG 2.1 accessibility compliance checks
 - Embed accessibility rules directly into the generation workflow.
 - Ensure that generated React forms include features such as semantic roles, labels, alt-text, and contrast validation.

3. METHODOLOGY

3.1. System Overview

The proposed research system is designed as an integrated platform that automates form generation and validation across both frontend and backend layers while ensuring accessibility and usability. The platform as a whole receives schema definitions, synchronizes them with backend validation rules, and produces frontend code enriched with accessibility features. Within this system, the Frontend Generator and Visual Builder section focuses on generating React code from JSON Schema, maintaining consistency with backend data transfer objects (DTOs), enabling non-technical users to create and modify forms through a drag-and-drop interface, embedding accessibility compliance checks, and exploring the use of artificial intelligence for optimization.

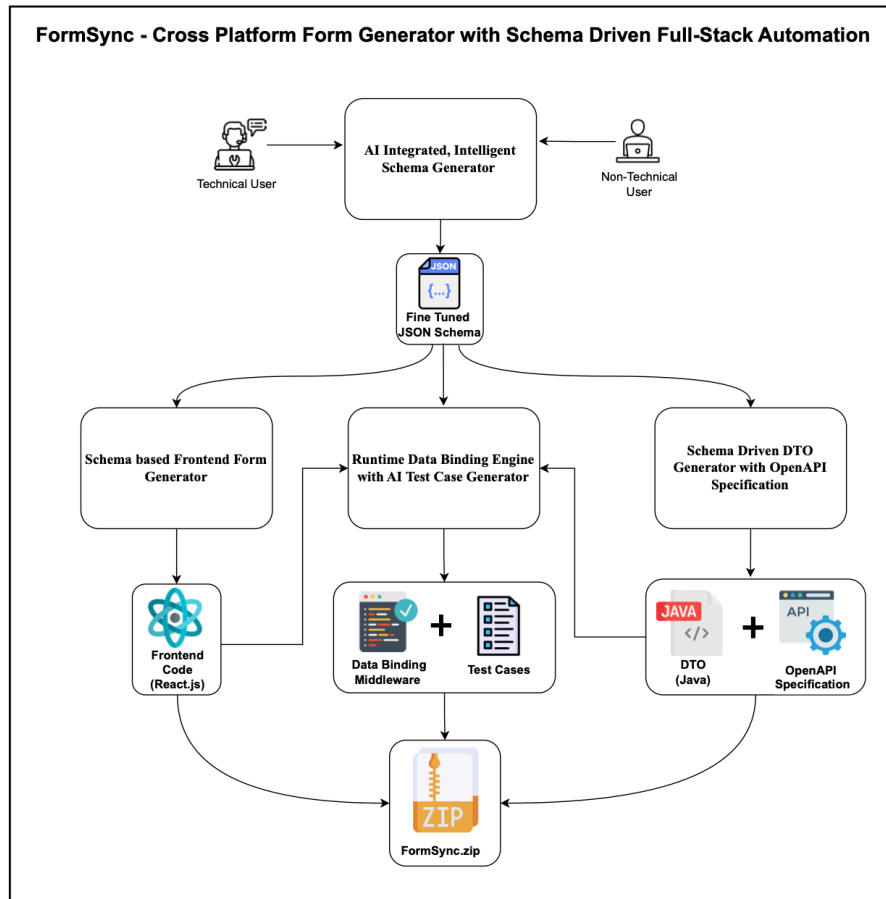


Figure 3.1 - System Overview Diagram

3.2. Core Processes and System Inputs

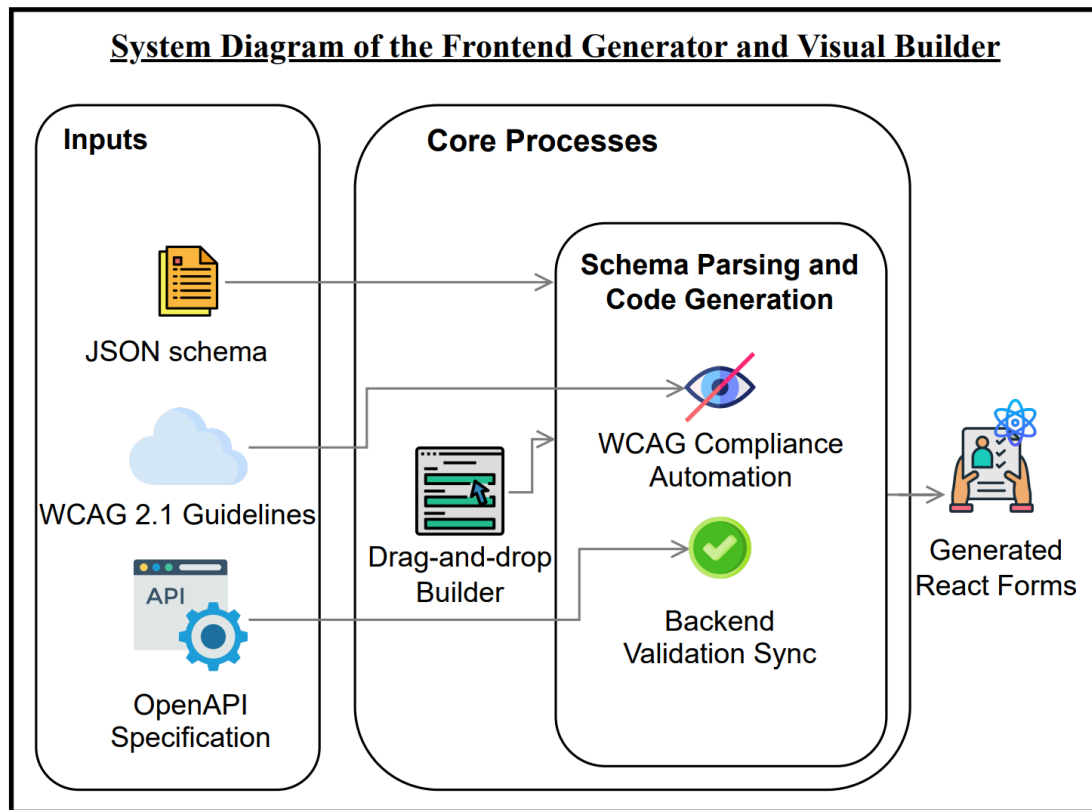


Figure 3.2 - Component System Overview Diagram

The frontend generator and visual builder section will operate primarily on three categories of input data. First, JSON Schema definitions supplied by the schema module will describe the structure of forms and the types of input fields required. Next backend DTO specifications from Spring Boot will provide validation annotations that define constraints such as required fields, size limits, or patterns. These rules will be extracted using standard parsing and reflection libraries rather than custom lexer-parser tools, ensuring efficiency and compatibility. Then finally, WCAG 2.1 guidelines will act as reference data for accessibility compliance, informing automated checks to guarantee that generated forms meet international standards.

- **Design and Implementation Steps** - The design and implementation of this module will proceed in five main steps, each aligned with the specific objectives.
- **Schema Parsing and React Code Generation** - The JSON Schema will be parsed and transformed into a structured representation that can be mapped to React

components. Validation logic will be generated using libraries such as Yup or Zod to ensure input constraints are enforced on the client side.

- **Backend Synchronization** - Validation annotations defined in backend DTOs will be extracted through reflection and annotation readers. These constraints will be merged with the JSON Schema before code generation to guarantee that frontend and backend rules remain consistent, minimizing duplication and preventing mismatches.
- **Drag-and-Drop Visual Form Builder** - A user-friendly interface will be developed to allow non-technical users to design and adjust forms through visual interaction. The builder will update the underlying schema dynamically and trigger regeneration of the React code. This ensures that users without programming expertise can contribute to form creation in a low-code or no-code manner.
- **WCAG 2.1 Compliance Automation** - Accessibility checks will be embedded in the generation process. Generated React components will include semantic roles, labels, alt-text attributes, proper contrast handling, and clear error feedback mechanisms. Automated evaluation tools will be integrated to validate compliance against WCAG 2.1 guidelines.

3.3. Tools, Frameworks, and Technologies

The frontend generator and visual builder will be implemented using React.js for code generation, supported by schema-based validation libraries such as Yup or Zod. JSON Schema parsers will be employed to interpret schema inputs, while Spring Boot reflection APIs will be used to capture backend validation constraints. Accessibility enforcement will rely on WCAG validation libraries and automated accessibility checkers. Artificial intelligence capabilities may be supported through frameworks such as TensorFlow or PyTorch, together with heuristic optimization algorithms. OpenAPI specifications may also be incorporated where available to enhance backend integration.

3.4. Evaluation and Testing Approach

Evaluation will consider functional, accessibility, usability, and performance aspects.

- Functional testing will verify that JSON Schemas are correctly translated into React code and that validation rules align with backend DTO constraints.
- Accessibility testing will be conducted using WCAG compliance checkers to ensure generated forms meet standards.
- Usability testing will evaluate the drag-and-drop builder with a small group of non-technical users to measure ease of use and learning curve.
- Performance testing will assess generation time and scalability when handling larger schemas.

3.5. Expected Outcomes

The methodology is expected to produce a working prototype of the Frontend Generator and Visual Builder that demonstrates schema-driven React code generation, synchronized backend validation, a user-friendly drag-and-drop interface, AI-assisted optimization, and automated accessibility enforcement. These outcomes will directly contribute to the overall goal of creating a unified platform where frontend generation, backend synchronization, usability, and compliance are managed in one place.

4. Functional and Non-Functional Requirements

4.1. Functional Requirements

The system must deliver the following functional capabilities to support schema-driven frontend form generation and accessibility compliance:

- **Schema-Driven Form Generation** - Generate React form components directly from JSON Schema definitions.
- **Backend Validation Synchronisation** - Extract validation rules from backend DTOs (Spring Boot) and ensure they are reflected in the frontend.
- **Drag-and-Drop Form Builder** - Provide a visual interface that allows non-technical users to create and configure forms.
- **Accessibility Compliance** - Embed WCAG 2.1 standards into generated forms, ensuring ARIA roles, semantic labels, and error feedback mechanisms are automatically included.
- **Code Export and Integration** - Enable developers to export production-ready React code that integrates into existing applications.
- **Validation Feedback** - Ensure form inputs provide real-time validation feedback to users in alignment with backend rules.
- **Schema Update Propagation** = Automatically regenerate form components when schema definitions are updated.

4.2. Non-Functional Requirements

In addition to functional aspects, the system should satisfy the following non-functional requirements to ensure performance, usability, and maintainability:

- **Performance** - Form generation should occur within seconds even for large schemas, ensuring smooth user interaction.
- **Scalability** - The system should support increasing numbers of schemas, forms, and users without degradation in performance.
- **Usability** - The drag-and-drop builder should be intuitive, requiring minimal learning for non-technical users.
- **Accessibility** - Generated forms must meet WCAG 2.1 AA compliance by default.
- **Maintainability** - Codebase should follow modular architecture, allowing future enhancements and integration with other frameworks.
- **Security** - Input validation mechanisms must prevent common vulnerabilities such as injection and cross-site scripting (XSS).
- **Portability** - Generated forms should be deployable across different environments (local, staging, cloud).
- **Reliability** - The system should consistently generate valid React code without breaking schema constraints.

5. GANTT CHART



Figure 5.1 - Gantt Chart for Schema-Based Frontend Builder

6. WORK BREAKDOWN STRUCTURE

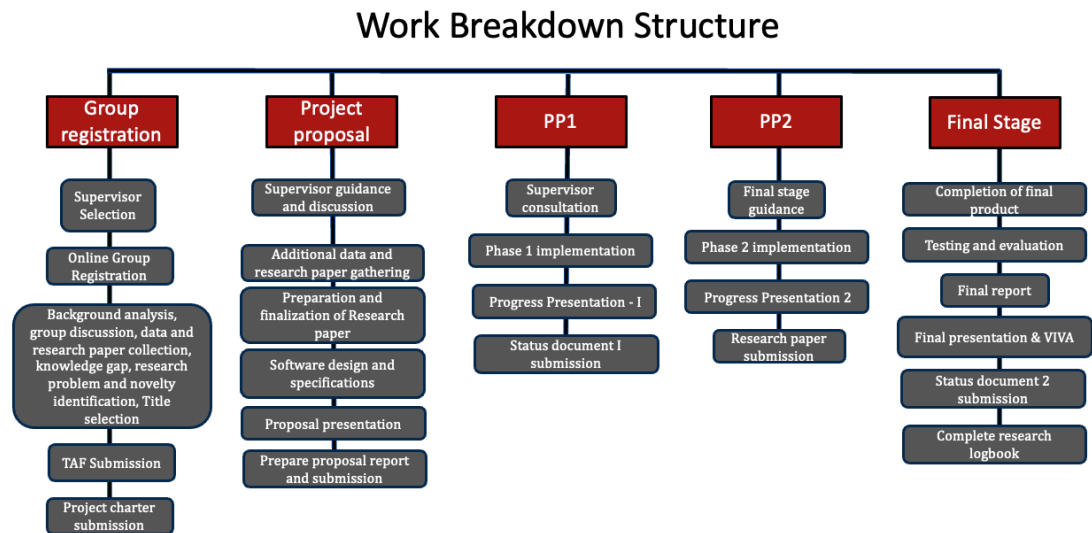


Figure 6.1 - Work Breakdown Structure for Schema Based Frontend Builder

7. COMMERCIALISATION AND ENTREPRENEURSHIP POTENTIAL

7.1. Market Review and Target Users

The demand for form generation tools is visible in both developer and enterprise contexts. Developers currently rely on libraries such as Formik or React Hook Form, which require repeated configuration and manual validation. Enterprises face the additional burden of ensuring accessibility compliance, often managed separately from development. A schema-driven generator addresses both needs by reducing duplication for developers while producing code that meets accessibility standards by default. The primary users are professional software development teams, enterprise IT departments, and organisations that need to enforce compliance across their digital services. A secondary user group consists of non-technical staff who can benefit from the visual builder for rapid prototyping and configuration.

7.2. Value Proposition

The proposed system offers a unique combination of features not present in current solutions. Developers gain from a single source of truth using JSON Schema that synchronises frontend and backend validation, removing error-prone duplication. Teams benefit from automated accessibility enforcement, reducing the risk of non-compliance. The visual drag-and-drop builder lowers the entry barrier, enabling non-technical stakeholders to participate in form design while still producing production-ready React code. Together, these aspects provide efficiency, reliability, and inclusivity in one workflow.

7.3. Business Model Approach

A Software-as-a-Service (SaaS) delivery model is proposed. The platform can be offered through subscription tiers, with basic access for individual developers, advanced features for small teams, and enterprise packages with integration and support options. This approach ensures predictable revenue streams and makes the service accessible to a wide range of organisations without heavy upfront costs. The

SaaS model also allows continuous updates, including improvements in accessibility standards, security patches, and extended framework support.

7.4. Cost and Resource Structure

The cost structure of the system primarily involves cloud infrastructure for hosting the service, ongoing development and maintenance of the code generator, and customer support. Additional resources include compliance testing tools, licensing of third-party libraries where necessary, and marketing activities. Compared to traditional licensed software, the SaaS model spreads costs across a recurring revenue base, making it sustainable with a moderate user adoption rate.

7.5. Commercial Viability Factors

Several factors support the commercial viability of this system. First, the schema-driven approach is already familiar to developers, making adoption easier. Second, the tool addresses compliance requirements that are often mandatory for public-facing organisations, creating a clear value driver. Third, integration with existing development environments ensures that the system does not disrupt workflows but instead complements them. These strengths increase the likelihood of acceptance by professional teams and enterprises.

7.6. Marketing and Adoption Strategy

Marketing efforts can focus on both developer communities and enterprise technology decision-makers. For developers, open access to a limited free tier and integration with widely used platforms such as GitHub can encourage adoption. For enterprises, demonstrations of compliance and productivity gains can serve as key selling points. Partnerships with consulting firms and training providers may also support adoption by reducing resistance to process change.

8. BUDGET AND BUDGET JUSTIFICATION

Category	Item	Cost (LKR)	Justification
Internet & Connectivity	University/ Home WiFi Access	0 LKR	High-speed campus internet is sufficient for development and collaboration.
Software & Services	Premium Development Tools (JetBrains/WebStorm, Debugging Plugins)	5,000 LKR	Required for efficient frontend coding and debugging.
Accessibility Testing Tools (axe-core, Lighthouse Premium)	4,000 LKR	Automated WCAG 2.1 compliance and accessibility validation.	
Cloud Deployment	AWS/Azure/GCP Credits	12,000 LKR	Hosting SaaS frontend generator, CI/CD pipeline.
Database Hosting (PostgreSQL)	4,000 LKR	Storage of form schemas, validation metadata.	
Load Balancer & CDN Services	3,000 LKR	Ensures scalable form builder performance and global access.	
Data & Resources	Research Paper Access (IEEE/ACM)	0 LKR	Covered by university library subscriptions.
Testing & Validation	User Testing Platform (Premium)	2,500 LKR	Usability testing with non-technical users for drag-and-drop builder.
Performance Testing Tools (JMeter/Artillery)	2,500 LKR	Load testing of form generation and rendering speed.	
Contingency (10%)	Emergency Fund	3,300 LKR	Coverage for unforeseen expenses (e.g., extra hosting, subscriptions).
TOTAL PROJECT COST	36,300 LKR	-	

Table 8.1 - Budget Justification

9. REFERENCES

- [1] Formik, "Formik Docs Overview," [Online]. Available: <https://formik.org/docs/overview>. [Accessed 07 07 2025].
- [2] React Hook Form, "React Hook Form," [Online]. Available: <https://react-hook-form.com>. [Accessed 16 08 2025].
- [3] JSONForms, "JSON Forms Docs," [Online]. Available: <https://jsonforms.io/docs/>. [Accessed 16 08 2025].
- [4] T. Beltramelli, "pix2code: Generating Code from a Graphical User Interface Screenshot," arXiv, 2017.
- [5] Y. C. J. L. L. C. L. S. T. Z. Shuhong Xiao, "Prototype2Code: End-to-end Front-end Code Generation from UI Design Prototypes," in Proceedings of the ASME 2024 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Washington, DC, 2024.
- [6] L. Y. T. Z. S. F. S. Yifei Duan, "Automated UI Interface Generation via Diffusion Models: Enhancing Personalization and Efficiency," in 2025 4th International Symposium on Computer Applications and Information Technology (ISCAIT), Xi'an, China, 2025.
- [7] C. C. Sabrina Haque, "Early Accessibility: Automating Alt-Text Generation for UI Icons During App Development," arXiv, 2025.
- [8] Uniforms, "Uniforms React Form Library," [Online]. Available: <https://uniforms.tools>. [Accessed 16 08 2025].
- [9] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: <https://www.w3.org/TR/WCAG21/>. [Accessed 10 08 2025].

- [10] O. Generator, "OpenAPI Generator Documentation," [Online]. Available: <https://openapi-generator.tech>. [Accessed 16 08 2025].
- [11] w3, "All WCAG 2.1 Understanding Docs," [Online]. Available: <https://www.w3.org/WAI/WCAG21/Understanding/>. [Accessed 10 08 2025].
- [12] T. S. G. M. Z. X. Y. L. Chunyang Chen, "From UI Design Image to GUI Skeleton: A Neural Machine Translator to Bootstrap Mobile GUI Implementation," in ACM/IEEE 40th International Conference on Software Engineering, 2018.
- [13] S. S. P. G. M. E. I. T. E. A. S. M. D. Batuhan Asiroglu, "A Deep Learning Based Object Detection System for User Interface Code Generation," in International Congress on Human-Computer Interaction, Optimization and Robotic Applications , 2022.
- [14] L. F. T. S. L. M. Y. L. L. X. Sen Chen, "Automated Cross-Platform GUI Code Generation for Mobile Apps," in 2019 IEEE 1st International Workshop on Artificial Intelligence for Mobile (AI4Mobile), Hangzhou, China, 2019.
- [15] K. B. F. M. Oksana Nikiforova, "Automated Generation of Web Application Front-end Components from User Interface Mockups," in 19th International Conference on Software Technologies, 2024.
- [16] D. L. Q. J. Yanming Zhang, "Visual Dynamic Form Model Generation Engine under a Multi-FrontEnd Framework," in International Conference on Computer, Artificial Intelligence, and Control Engineering (CAICE 2023), 2023.
- [17] G. M. K. P. M. Ibrahim Althomali, "Automated Repair of Responsive Web Page Layouts," in 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), 2022.

- [18] P. M. B. R. B. S. Usharani, "Voice Based Form Filling System For Visually Challenged People," in 2020 International Conference on System, Computation, Automation and Networking (ICSCAN), Pondicherry, India, 2020.
- [19] R. S. S. R. P. K. B. C. P. S. S. L. N. D. & G. S. Adarsh R. Hegde, "Automated government form filling for aged and monolingual people using interactive tool," *Disability and Rehabilitation: Assistive Technology*, vol. 19, no. 6, pp. 2260-2270, 2024.
- [20] Y. S. M. Z. Y. Z. T. W. H. H. S. H. G. H. D. Zhang, "FormLM: Recommending Creation Ideas for Online Forms by Modelling Semantic and Structural Information," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates, 2022.
- [21] G. B. Robert Waszkowski, "Visibility Matrix: Efficient User Interface Modelling for Low-Code Development Platforms," *Sustainability*, vol. 14, no. 13, p. 8103, 2022.
- [22] J. R. Jung-woo Sohn, "Securing web applications with better “patches”: an architectural approach for systematic input validation with security patterns," in *2015 10th International Conference on Availability, Reliability and Security (ARES)*, Toulouse, France, 2015.
- [23] S. W. Y. Z. T. Y. Zhiguo Ding and Z. L. H. S. Y. S. Yingna Li, "CodingCare: AI Code Generation Security Framework for Common Vulnerability Mitigation," in *ISCCN 2025: 2025 4th International Conference on Intelligent Systems, Communications and Computer Networks*, Nanjing China, 2025.